

Embedded Vision Intelligent Laboratory

微算機系統實習

MICROPROCESSOR SYSTEMS LAB.

AUTUMN, 2026

Instructor：Yen-Lin Chen(陳彥霖), Ph.D.

Professor

Dept. Computer Science and Information Engineering

National Taipei University of Technology

作業評分方式

- 實驗部分佔該次實驗的總比例70%
  小組報告總分佔該次實驗的總比例15%
  個人報告總分佔該次實驗的總比例15%
  ＊當次實驗會因為難易度不同佔學期總成績的實驗分數比例也不同
- 實驗報告上傳格式，公告於i學園
- 實驗報告以小組為單位繳交報告

請同學上傳報告時依照上面的格式上傳:

- 一、組別與組員名單
- 二、實驗步驟截圖與說明
- 三、組員貢獻比例（組員%數加總必須等於100%）
- 四、個人心得（額外單獨繳交）

注意事項:

- 實驗報告單獨分數為100分，以上第一、三、四點每缺少一項報告分數扣20分。第二點缺少報告扣40分。另外第二、四點不完整會依照狀況扣分，最高扣到該項目的上限分數。
- 貢獻比例分配方式為: 報告總分*人數*組員貢獻比例=組員報告得分

作業繳交

- 基本繳交時間
  -實驗、報告：
  - 實驗驗收:公布實驗後的下個星期三(10/28)下課結束(16:00)前
  - 報告:實驗驗收後的下星期三(11/4)結束前(23:59)



＊若有因為特殊原因繳交時間有變動助教會另外公布
超過時間遲交每隔一週（含一週內）分數打8折，採累計連乘方式，實驗與報告打折是分開算的，

最多遲交兩週，<遲交的項目單獨分數>　*　0.64

- 舉例：
  遲交三天－以遲交一週計算 <遲交的項目單獨分數>*0.8＝該項目得到的分數
  遲交九天－以遲交兩週計算 <遲交的項目單獨分數>*0.8*0.8＝該項目得到的分數

- 以上配分與注意事項有問題請聯絡助教

報告評分方式

- 請依照雲端和I學園公告欄的評分標準為準
  https://drive.google.com/drive/folders/1Ywev6PvLS7s1W_UtmpFZLUWIAln3sTqk?usp=sharing

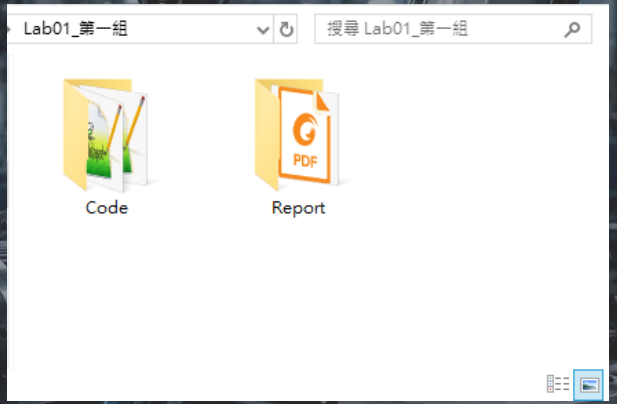
作業繳交格式

- **請確實依照規定的檔案名稱、副檔名上傳，否則不予計分**
- 小組、個人報告檔案格式：
  - **報告本身使用格式(.pdf、.docx、.odt)擇一上傳**
  - **Mac用戶限定.pdf上傳，未依此規定檔案格式上傳者視同未繳交**
    - 小組 ： 使用”Lab<number>_第<number>組_小組報告” 命名。
    - 個人 ： 使用” Lab<number>_第<number>組_<學號>_<姓名>_個人報告” 命名。
  - 資料夾請使用” Lab<number>_第<number>組” 命名，並壓縮為**.zip**檔
  - <number>部分請填零到兩位數，ex: Lab01⭕、Lab1❌
  - 母資料夾裡要包含如下子資料夾
    - -Code //存放專案程式碼
    - -Report //存放報告
    - -若需要拍攝影片，請上傳至雲端空間(ex:Youtube)，附上連結網址、影片截圖至小組報告內。請確保連結網址可以直接觀看影片。

作業繳交格式 cont.

- 小組報告繳交範例:

```
.
└── Lab01_第01組.zip
    └── Lab01_第01組
        ├── Code
        │   ├── main.cpp
        │   └── test.cpp
        └── Report
            └── Lab01_第01組_小組報告.pdf
```



- 個人報告繳交範例:

```
.
└── Lab01_第01組_111ABC001_陳小明_個人報告.pdf
```

實驗五



SPI控制實習

注意事項

- 請每次實驗都要確保「借」跟「還」板子的時候要找助教登記
- 每次要外借跟續借時請找助教「簽名畫押」以示負責
- 如助教在清點時未能找到板子，以及未出現登記歸還、外借或續借

後果自負



注意事項

- 在開始實驗前，請先詳細閱讀實驗步驟
- 檢查前請先看看自己組別是否都達成評分項目的要求
- 不懂的地方先去找 Lecture 1~5和Lab 5的簡報，真的不理解再去找助教





實驗說明

本次實驗目標

- 學習如何應用麵包板結合感測器與LED進行資料擷取
- 學習如何透過GPIO與Python讀取TX2上的光敏電阻數值。
- 學習如何透過GPIO與Python控制TX2上的LED燈。
- 了解TX2 physical pin對應的sysfs filename
- 結合感測器與LED進行實驗
- 學習如何使用TX2上的CSI接口的相機

LAB 5 項目要求

透過Python控制GPIO上的光敏電阻和2個LED

(此頁項目一和項目二請寫成同一個程式)

- **項目一：讀取光敏電阻數值(20%)**
  - 在麵包板上安裝類比數位訊號轉換器及光敏電阻
  - 撰寫Python讀取光敏電阻數值
  - 在Terminal上顯示光敏電阻數值
- **項目二：設定門檻值控制LED燈狀態(10%)**
- 透過讀取光敏電阻數值設定門檻值
  - 依照設定的門檻值控制2顆LED燈開關狀態(門檻值自行設定)
  - 在Terminal上顯示LED燈狀態

LAB 5 項目要求

- **項目三：測試CSI Camera (10%)** **項目三為command，不用另寫程式**
  - 使用TX2上之CSI相機，並顯示畫面
  - 螢幕擷取乙張影像附在報告中
  - P.S.(同組可愛合照or錄影)佳

- **項目四:防盜快照系統(30%)**
  - 1.計算目前與前次的ADC Value差值，使光照變化差值超過設定值視為異常狀態。(設定差值跨度不要太小)
  - 2.異常狀態事件:
  - 閃爍警報:異常狀態下兩顆LED交互閃爍
  - 即時拍照:相機啟動，連續拍下3張照片
  - 照片須以當下時間自動命名並歸檔，隨後使系統復歸
  - 在Terminal上顯示目前與前次的ADC Value與差值，以及事件紀錄(異常狀態、拍照)
  - 持續異常監測：初次拍照完成後，系統持續追蹤監測即時 ADC 數據。若狀態依然異常（即「即時數值」與「進入異常前的原始基準值」之差值仍大於門檻），則持續進行追拍與警報閃爍。
  - 系統復歸：直到即時監測數據回歸並接近觸發前的原始數值（兩者差值小於門檻）時，系統才關閉相機、熄滅 LED，正式復歸到正常狀態

LAB6 總共有2個.py程式



實驗器材

器材領取

每組派一位同學領取實驗器材，包含：

1.杜邦線公對公*7
2.杜邦線公對母*15
3.小麵包板*1
4.光敏電阻*1
5.MCP3008*1(針腳脆弱，請小心使用)
6.USB 相機 (若TX2上無CSI Camera)

期末實驗結束需回收，請同學們愛惜器材



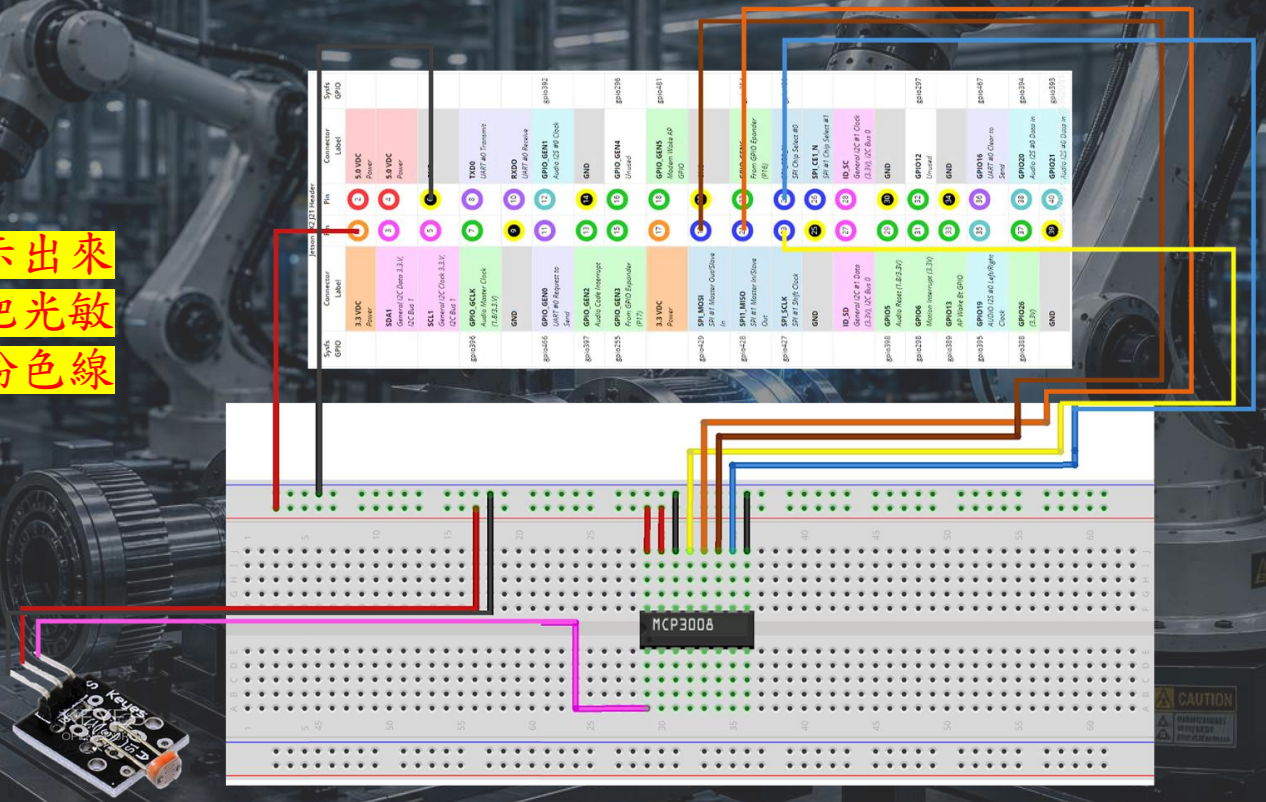
透過Python控制gpio

RPI.GPIO模組腳位對照表

wiringPi 编码	BCM 编码	功能名	物理引脚 BOARD编码		功能名	BCM 编码	wiringPi 编码
		3.3V	1	2	5V		
8	2	SDA.1	3	4	5V		
9	3	SCL.1	5	6	GND		
7	4	GPIO.7	7	8	TXD	14	15
		GND	9	10	RXD	15	16
0	17	GPIO.0	11	12	GPIO.1	18	1
2	27	GPIO.2	13	14	GND		
3	22	GPIO.3	15	16	GPIO.4	23	4
		3.3V	17	18	GPIO.5	24	5
12	10	MOSI	19	20	GND		
13	9	MISO	21	22	GPIO.6	25	6
14	11	SCLK	23	24	CE0	8	10
		GND	25	26	CE1	7	11
30	0	SDA.0	27	28	SCL.0	1	31
21	5	GPIO.21	29	30	GND		
22	6	GPIO.22	31	32	GPIO.26	12	26
23	13	GPIO.23	33	34	GND		
24	19	GPIO.24	35	36	GPIO.27	16	27
25	26	GPIO.25	37	38	GPIO.28	20	28
		GND	39	40	GPIO.29	21	29

轉換器與光敏電阻接法（以BCM編碼方式為例）

如果數值顯示出來有問題，請把光敏電阻的黑和粉色線交換



RPI.GPIO模組介紹

- GPIO.setmode():RPI有多個編號系統，在程式設計之前需要指定一個編號系統
- GPIO.setup():該腳位需要作為輸入還是輸出
- GPIO.output():設定輸出電位高低
- GPIO.input():在某個時間點檢查輸入值
- GPIO.cleanup():清除正在使用的腳位編號系統

MCP3008訊號讀取方式

- input一次送三個bytes
  - # byte 1: the start bit (always 0x01)
  - # byte 2: configure bits
  - # byte 3: don't care

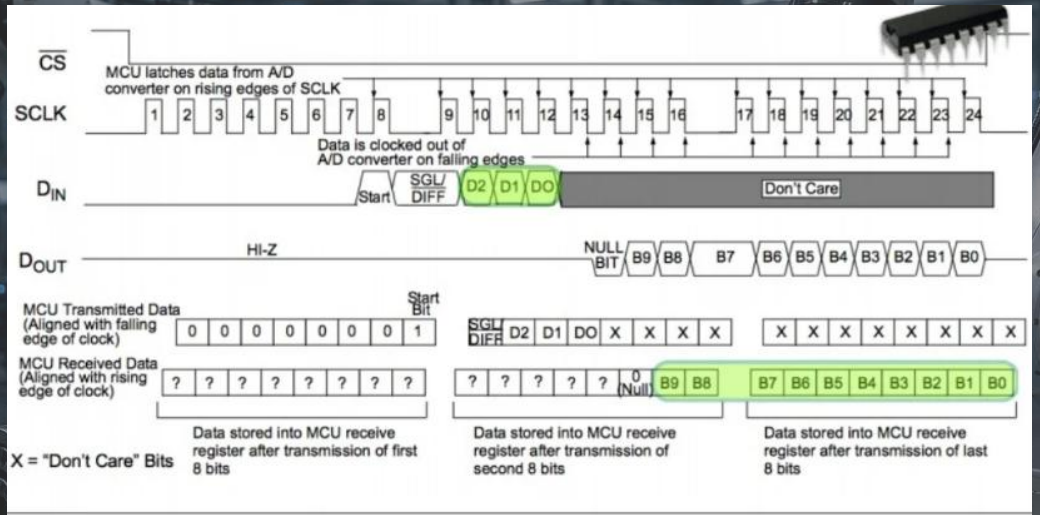
0x01 Ch0/Ch1 don't care

- spi.xfer2([1,(8+channel)<<4,0])
  - Ch0 = 1000 0000
  - Ch1 = 1001 0000

TABLE 5-2: CONFIGURE BITS FOR THE MCP3008

Control Bit Selections				Input Configuration	Channel Selection
Single /Diff	D2	D1	D0		
1	0	0	0	single-ended	CH0
1	0	0	1	single-ended	CH1
1	0	1	0	single-ended	CH2
1	0	1	1	single-ended	CH3
1	1	0	0	single-ended	CH4
1	1	0	1	single-ended	CH5
1	1	1	0	single-ended	CH6
1	1	1	1	single-ended	CH7

MCP3008訊號讀取方式




```
IN =[0000 0001][1CNL ----][---- ----]
                              (8+channel) <<4
OUT=[---- ----][---- -XXX][XXXX XXXX]  (10bit)
                    ((r[1] & 3) << 8) + r[2]
```

Python腳位設定

```
import Jetson.GPIO as GPIO
import time


# change these as desired - they're the pins connected from the
# SPI port on the ADC to the Cobbler
SPICLK = 11
SPIMISO = 9
SPIMOSI = 10
SPICS = 8
output_pin = 7
# photoresistor connected to adc #0
photo_ch = 0

#port init
def init():
  #GPIO.BOARD   GPIO.BCM  GPIO.CVM  GPIO.TEGRA_SOC
        GPIO.setmode(GPIO.BOARD)
        GPIO.setup(output_pin, GPIO.OUT, initial=GPIO.HIGH)
        GPIO.setwarnings(False)
        GPIO.cleanup()   #clean up at the end of your script
        GPIO.setmode(GPIO.BCM)  #to specify whilch pin numbering system
        # set up the SPI interface pins
        GPIO.setup(SPIMOSI, GPIO.OUT)
        GPIO.setup(SPIMISO, GPIO.IN)
        GPIO.setup(SPICLK, GPIO.OUT)
        GPIO.setup(SPICS, GPIO.OUT)
```

SPI 腳位位置(對應板子上的腳位)

MCP3008類比轉數位腳位編號(0~7,對應轉換器上腳位)

腳位初始設定

Python讀取感測器

```
#read SPI data from MCP3008(or MCP3204) chip,8 possible adc's (0 thru 7)
def readadc(adcnum, clockpin, mosipin, misopin, cspin):
        if ((adcnum > 7) or (adcnum < 0)):
                return -1
        GPIO.output(cspin, True)

        GPIO.output(clockpin, False)  # start clock low
        GPIO.output(cspin, False)     # bring CS low

        commandout = adcnum
        commandout |= 0x18  # start bit + single-ended bit
        commandout <<= 3    # we only need to send 5 bits here
        for i in range(5):
                if (commandout & 0x80):
                        GPIO.output(mosipin, True)
                else:
                        GPIO.output(mosipin, False)
                commandout <<= 1
                GPIO.output(clockpin, True)
                GPIO.output(clockpin, False)

        adcout = 0
        # read in one empty bit, one null bit and 10 ADC bits
        for i in range(12):
                GPIO.output(clockpin, True)
                GPIO.output(clockpin, False)
                adcout <<= 1
                if (GPIO.input(misopin)):
                        adcout |= 0x1

        GPIO.output(cspin, True)

        adcout >>= 1       # first bit is 'null' so drop it
        return adcout
```

GPIO -> ADC
ADC讀入類比訊號

ADC -> GPIO
ADC輸出數位訊號

Python讀取感測器

```
#read SPI data from MCP3008(or MCP3204) chip,8 possible adc's (0 thru 7)
def readadc(adcnum, clockpin, mosipin, misopin, cspin):
    if ((adcnum > 7) or (adcnum < 0)): # 檢查 ADC 通道編號是否在有效範圍內，如果超出範圍則返回 -1。
        return -1
    GPIO.output(cspin, True) # 將cspin設置為高電位，表示開始與 ADC 通訊。

    GPIO.output(clockpin, False) # start clock low # 將clockpin設置為低電位，確保通訊起始時時鐘訊號為低。
    GPIO.output(cspin, False) # bring CS low # 將cspin設置為低電位，表示開始向 ADC 發送指令。

    commandout = adcnum  # 將 ADC 通道編號保存到命令變數中。
    commandout |= 0x18 # start bit + single-ended bit # 對commandout進行按位或運算，將 0x18（0b00011000）的值加到命令變數中。
    commandout <<= 3 # 只會用到 5 bits # 將commandout向左移動 3 位，0b11000 000。
    for i in range(5):
        if (commandout & 0x80):# 檢查commandout的最高位是否為 1。0x80(0b10000000)&commandout
            GPIO.output(mosipin, True) # 如果最高位為 1，則將 MOSI 腳位設置為高電位，表示向 ADC 發送 1。
        else:
            GPIO.output(mosipin, False) # 如果最高位不為 1，則將 MOSI 腳位設置為低電位，表示向 ADC 發送 0。
        commandout <<= 1 # 將commandout向左移動 1 位。0b11000 000->0b10000 000
        GPIO.output(clockpin, True) # 將clockpin設置為高電位，觸發正緣(rising edge)。
        GPIO.output(clockpin, False)  # 將clockpin設置為低電位，觸發負緣(falling edge)。
    adcout = 0  # 初始化讀取的 ADC 數值為 0。
    # read in one bit, one null bit and 10 ADB bits.
    for i in range(12):
        GPIO.output(clockpin, True) # 將clockpin設置為高電位，觸發正緣(rising edge)。
        GPIO.output(clockpin, False) # 將clockpin設置為低電位，觸發負緣(falling edge)。
        adcout <<= 1 # 將讀取的 ADC 數值向左位移 1 位，為下一個位元組的接收做準備。
        if (GPIO.input(misopin)): # 檢查 MISO 腳位的數值。
            adcout |= 0x1 # 如果 MISO 腳位的數值為高電位（1），則將 adcout 的最低位設置為 1。
    GPIO.output(cspin, True)  # 將cspin設置為高電位，表示結束與 ADC 的通訊。

    # first bit is 'null' so drop it.
    adcout >>= 1
    return adcout
```

GPIO -> ADC

ADC讀入類比訊號

ADC -> GPIO

ADC輸出數位訊號

Python讀取感測器

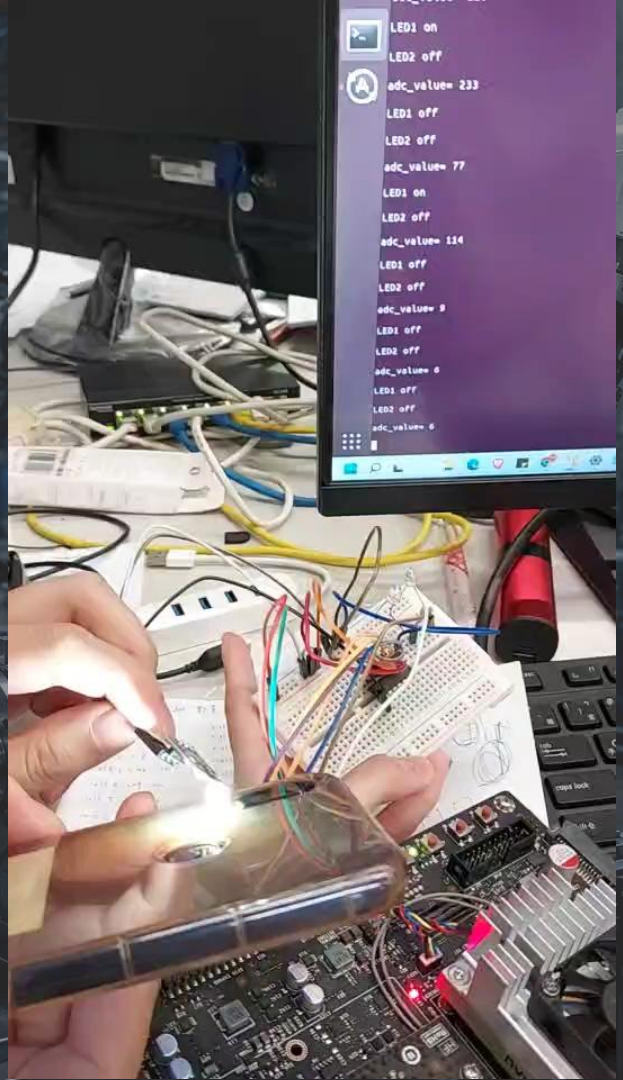
```
def main():
        init()
        while True:
                adc_value=readadc(photo_ch, SPICLK, SPIMOSI, SPIMISO, SPICS)
                print(adc_value)
                time.sleep(1)


if __name__ == '__main__':
        try:
                main()
```

執行結果

- 在TX2上執行Python檔案
- 終端機上顯示當前光敏電阻數值
- 當數值達到設定門檻值後LED燈亮
- 終端機上顯示LED燈亮狀態

範例影片


LED1 on
LED2 off
adc_value= 233
LED1 off
LED2 off
adc_value= 77
LED1 on
LED2 off
adc_value= 114
LED1 off
LED2 off
adc_value= 8
LED1 off
LED2 off
adc_value= 6
LED1 off
LED2 off
adc_value= 6

測試CSI Camera



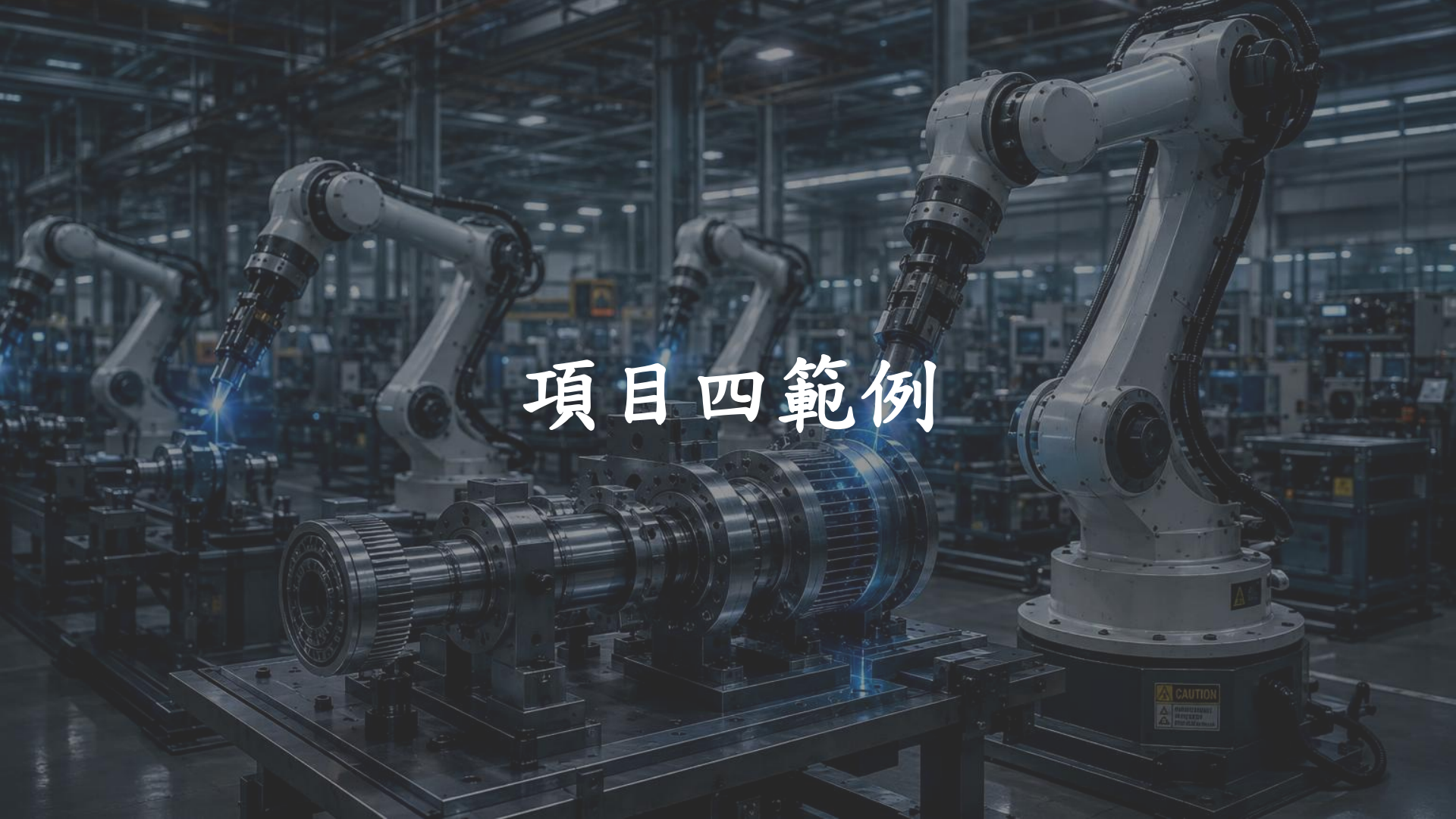
程式碼

- #測試相機
- gst-launch-1.0 nvarguscamerasrc ! nvvidconv ! xvimagesink

#如果失敗換下面的舊版本指令

- gst-launch-1.0 nvcamerasrc ! 'video/x-raw(memory:NVMM),width=640, height=480, framerate=30/1, format=NV12' ! nvvidconv flip-method=2 ! nvegltransform ! nveglglessink –e

- 實做前請使用 ssh –Y 參數 ex. ssh –Y nvidia@129.168.55.1



項目四範例

在python中使用opencv調用gstreamer

定義函式與預設參數

nvarguscamerasrc

• sensor_id={sensor_id}：指定哪一個鏡頭• video/x-raw(memory:NVMM)：將相機畫面直接放進 Jetson 的 NVMM處理
width,height相機的解析度（ format=(string)NV12：原始影像色彩格式•
framerate=(fraction){framerate}/1:幀率（FPS）。

nvvidconv

• flip_method={flip_method}：畫面旋轉/翻轉方式。
• video/x-raw：將影像傳回系統主記憶體（RAM）
• width=(int){display_width}, height=(int){display_height}：最終顯示/處理的解析度。

videoconvert

video/x-raw, format=(string)BGR：轉為標準 BGR 格式

appsink

drop=True：丟棄舊影格
sync=False：不同步時間戳記

在python中使用opencv調用gstreamer

請建立一個名為 gstreamer_pipeline 的函式。這個函式必須包含以下 7 個參數，並給予它們預設值：

- 攝影機編號：sensor_id（預設為 0）
- 擷取寬度：capture_width（預設為 1280）
- 擷取高度：capture_height（預設為 720）
- 顯示寬度：display_width（預設為 1280）
- 顯示高度：display_height（預設為 720）
- 幀率：framerate（預設為 30）
- 翻轉方法：flip_method（預設為 0）

在python中使用opencv調用gstreamer

在函式內部使用 return 。
請依序寫入以下五段 GStreamer 管道節點（記得每一行的末尾，驚嘆號！後面都要留一個空格）：

1. 第一行：設定 nvarguscamerasrc 節點，並留下一個整數格式化代號（%d）給攝影機編號。
2. 第二行：設定原始影像格式（video/x-raw(memory:NVMM)），並依序留下寬度（轉 int）、高度（轉 int）以及幀率（分數型態 fraction）的格式化代號。
3. 第三行：設定畫面翻轉節點（nvvidconv），留下翻轉方法的代號。
4. 第四行：設定轉換後的影像格式，依序留下顯示寬度（轉 int）、顯示高度（轉 int），並指定格式為 (string)BGRx。
5. 第五與第六行：進行色彩轉換（videoconvert），最終輸出為 OpenCV 可讀取的 (string)BGR 格式，並送入接收端 appsink。

在多行字串結束的括號右側，加上 % (...)。
請將你在步驟 1 定義的 7 個變數，嚴格依照字串中代號出現的先後順序填入括號中。順序如果錯了，解析出來的相機參數就會出錯！

在python中使用opencv調用gstreamer

```
import cv2
```

```
def gstreamer_pipeline(
    sensor_id=0,
    capture_width=1280,
    capture_height=720,
    display_width=1280,
    display_height=720,
    framerate=30,
    flip_method=0,
):
    return (
        "nvarguscamerasrc sensor-id=%d ! "
        "video/x-raw(memory:NVMM), width=(int)%d, height=(int)%d, framerate=(fraction)%d/1 ! "
        "nvvidconv flip-method=%d ! "
        "video/x-raw, width=(int)%d, height=(int)%d, format=(string)BGRx ! "
        "videoconvert ! "
        "video/x-raw, format=(string)BGR ! appsink"
        % (
            sensor_id,
            capture_width,
            capture_height,
            framerate,
            flip_method,
            display_width,
            display_height,
        )
    )
```

相機參數

在python中使用opencv調用gstreamer

請建立一個名為 pipeline_str 的變數，並將你剛剛的 gstreamer_pipeline 函式執行結果賦值給它。

- 在呼叫函式時，請主動傳入兩個參數的關鍵字：指定攝影機編號 sensor_id 為 0，以及畫面翻轉方法 flip_method 為 0。
- 思考：其他沒有傳入的參數（如寬度、高度），此時會帶入什麼數值？（提示：回想剛才設定的預設值）。

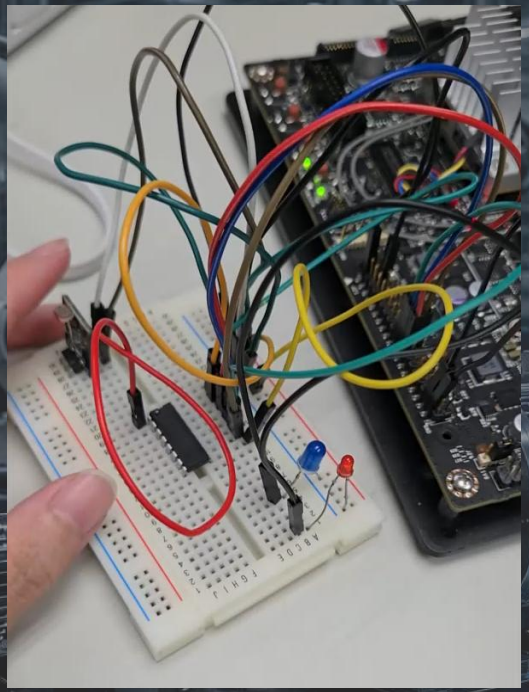
請建立一個名為 cap 的變數，並呼叫 OpenCV 讀取影像的類別 cv2.VideoCapture() 來進行初始化。

- 呼叫時必須傳入 2 個參數：
    1. 第一個參數：帶入你剛剛在步驟 1 建立的設定字串變數。
    2. 第二個參數：告訴 OpenCV 我們要使用哪一個底層後端驅動。請指定為 OpenCV 內建的 GStreamer 常數標籤。

在python中使用opencv調用gstreamer

```
# 使用 Jetson 專用 GStreamer 管線開啟 CSI 相機
pipeline_str = gstreamer_pipeline(sensor_id=0, flip_method=0)
cap = cv2.VideoCapture(pipeline_str, cv2.CAP_GSTREAMER)
```

呼叫 OpenCV 讀取影像



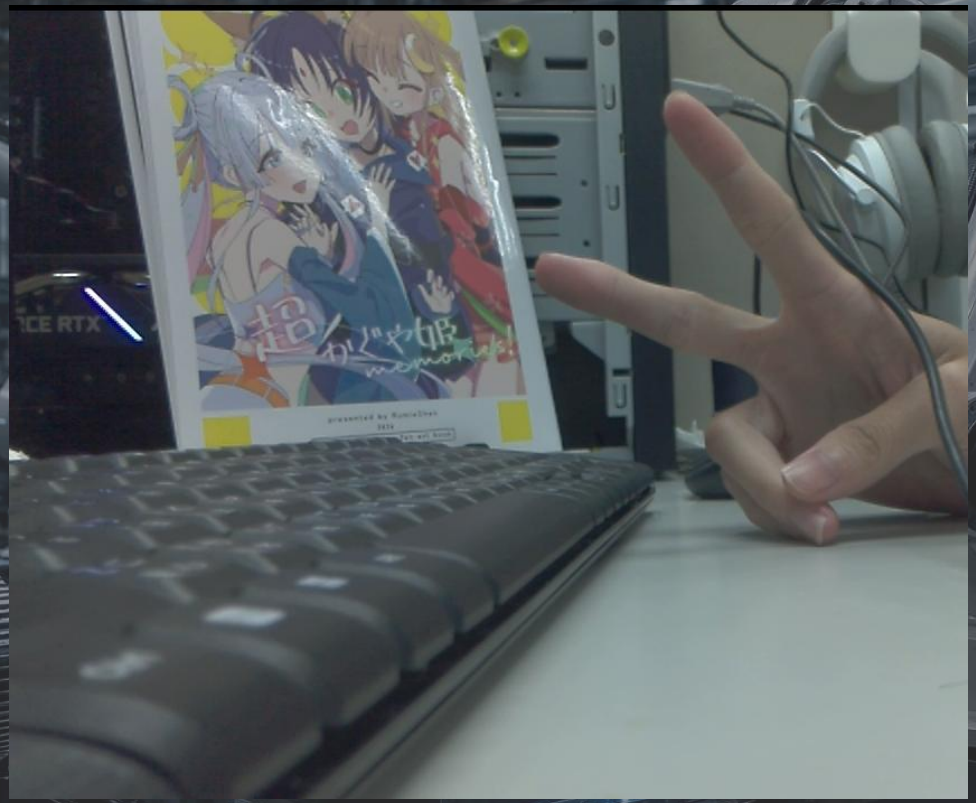
終端機顯示目前、前次ADC value 以及計算之差值

```
Current value: 219 | Previous value: 207 | Instantaneous change: 12
Current value: 205 | Previous value: 219 | Instantaneous change: 14
Current value: 175 | Previous value: 205 | Instantaneous change: 30
Current value: 274 | Previous value: 175 | Instantaneous change: 99
Current value: 162 | Previous value: 274 | Instantaneous change: 112
Current value: 302 | Previous value: 162 | Instantaneous change: 140
Current value: 583 | Previous value: 302 | Instantaneous change: 281
```

```
Abnormal Status

save to snapshots/intruder_20261019_102818_shot1.jpg
save to snapshots/intruder_20261019_102818_shot2.jpg
save to snapshots/intruder_20261019_102818_shot3.jpg
GST_ARGUS: Cleaning up
CONSUMER: Done Success
GST_ARGUS: Done Success
close camera
System Reset
```

當環境數據的瞬間差值超過額定值時，系統會立即進入異常狀態並自動拍攝照片存檔。隨後，系統將持續監控即時數據，若異常狀態未解除，則相機保持開啟並持續追拍，直到『即時數值』回歸並接近『觸發前的原始數值』（即兩者差值小於門檻）後，系統才執行復歸，回到正常狀態。


CAUTION

Q/A

- Q：為什麼要接IC？明明Arduino可以直接用
- A：因為TX2的GPIO沒有ADC
- Q：為什麼前面使用C++，但是現在要寫Python?
- A：官方用Python寫GPIO讀ADC的資料。
- A2：網上有基於官方Python套件庫編撰的非官方的C++版
- Q：為什麼LED都不明顯?
- A：好啦好啦!有買新的了



參考網址

- OpenCV 擷取網路攝影機串流影像，處理並寫入影片檔案教學
- use camera on jetson TX2
- How to use USB webcam in Jetson TX2 with Python and OpenCV?